

Palladium Bank Dimensional Model - Design Explanation

This data model was designed to support fast, consistent and business-focused reporting for Palladium Bank using 18 months of raw transaction data. The goal is to move away from querying raw logs directly and instead provide a structured analytics layer that supports customer behaviour analysis, product performance, branch monitoring and churn detection.

1. Fact Table Grain and Structure

The central table in this model is **fact_transactions**, designed at a **transaction-level grain**, meaning one row represents one financial transaction.

This decision is very important because:

- It preserves the full detail of customer activity
- It supports flexible aggregation (daily, monthly, customer-level, etc.)
- It enables accurate behavioural tracking like frequency and recency

The fact table stores key measures such as:

- transaction amount
- balance after transaction

It also includes **txn_id** as a **degenerate dimension**, since it has analytical value (tracking individual transactions) but does not require a separate dimension table.

2. Dimension Tables and Design Choices

Six dimension tables were created to give proper context to transactions:

- **dim_date**: supports time-based analysis (MoM, QoQ, YoY). A surrogate key (date_key) is used for performance and consistency.
- **dim_customer**: stores customer details and tier classification (Platinum, Gold, etc.). This is critical for identifying high-value customers and churn signals.
- **dim_branch**: captures branch location and enables regional performance analysis across Lagos, Abuja, Kano, Port Harcourt and Ibadan.
- **dim_product**: groups banking offerings (Savings, Loans, Cards, etc.) for product-level insights.

- **dim_channel:** shows how transactions happen (ATM, Mobile App, Branch, etc.), supporting digital vs physical channel analysis.
- **dim_transaction_type:** defines what the transaction represents (Deposit, Withdrawal, Transfer) and whether it is an inflow or outflow.

Each dimension uses a **surrogate key** instead of natural keys to:

- improve join performance
- handle historical changes cleanly
- avoid dependency on source system inconsistencies

3. Slowly Changing Dimensions (SCD)

Two SCD strategies were applied based on how the data behaves in real life.

SCD Type 2 – dim_customer

Customer tier can change over time (e.g. Silver to Gold). Instead of overwriting the old value, a new record is created with:

- effective_date
- expiry_date
- is_current flag

This allows the business to:

- track how customer value evolves
- analyse behaviour before and after tier changes
- detect early churn signals

SCD Type 1 – dim_branch

Branch details (like name or state) are not critical for historical tracking. If a change happens, the record is simply updated.

This keeps the model simple and avoids unnecessary data duplication.

4. ETL / ELT Strategy

The data loading process is divided into **initial load** and **incremental load**.

Initial Load

For the first 18 months of data:

1. Load dim_date first (no dependencies)
2. Load static dimensions (transaction type, channel)
3. Load branch, product and customer dimensions
4. Load fact_transactions last (after resolving all foreign keys)

During this process:

- natural keys are mapped to surrogate keys
- transformations are applied (e.g. classifying inflow/outflow)

Incremental Load

For new incoming data:

- New transactions are identified using txn_id
- Duplicate records are prevented using checks before insert
- Customer tier changes trigger the **SCD Type 2 procedure**
- Branch updates trigger the **SCD Type 1 overwrite**

This ensures the warehouse stays up to date without breaking historical accuracy.

5. Data Quality Checks

To maintain trust in the data, four key checks are enforced:

- **NULL or zero amounts:** flagged and excluded to prevent misleading totals
- **Duplicate transaction IDs:** removed to avoid double counting
- **Orphan records:** ensures every transaction links to valid dimensions
- **Invalid dates:** prevents future or out-of-range records from entering the system

These checks protect the integrity of reporting outputs.

6. Hierarchies and Analytics Support

The model supports drill-down analysis through defined hierarchies:

- Time: Year → Quarter → Month → Date
- Location: State → Branch
- Product: Product Type → Product Name

This allows users to move from high-level summaries to detailed insights easily.

7. Performance and Scalability

To ensure the model performs well as data grows:

- **Indexes** are applied on frequently queried columns like customer_key and date_key
- **Composite indexes** support common queries (e.g. branch + date analysis)
- **Partitioning** is recommended on date_key to reduce query scan size

An additional **aggregation table (agg_monthly_branch_revenue)** was created to:

- pre-calculate monthly totals
- reduce query time for dashboards
- improve performance for executive reporting

Conclusion

This star schema design transforms raw transaction data into a structured, scalable and analysis-ready system. By combining proper grain definition, SCD handling, strong ETL processes and performance optimisation, the model enables Palladium Bank to answer key business questions quickly and confidently.